

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ІВАНА ФРАНКА

ЗВІТ

з дисципліни “Функціональне програмування”

на тему « Основи функціонального програмування у Python »

Виконав:

Студент 2 курсу

групи ФсП-23

МАГОРА Максим

Перевірив:

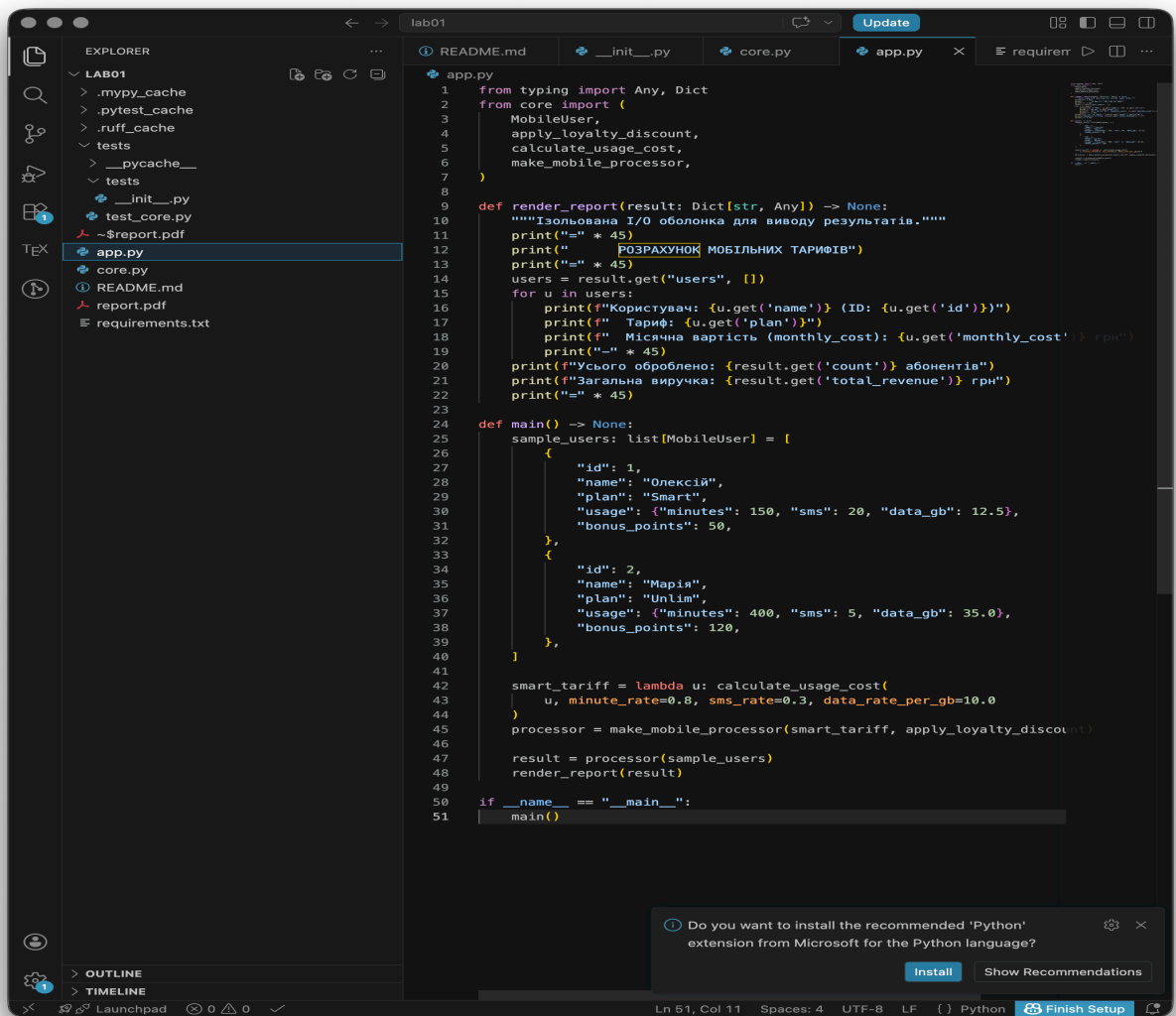
СВАЛЕБА Сергій Андрійович

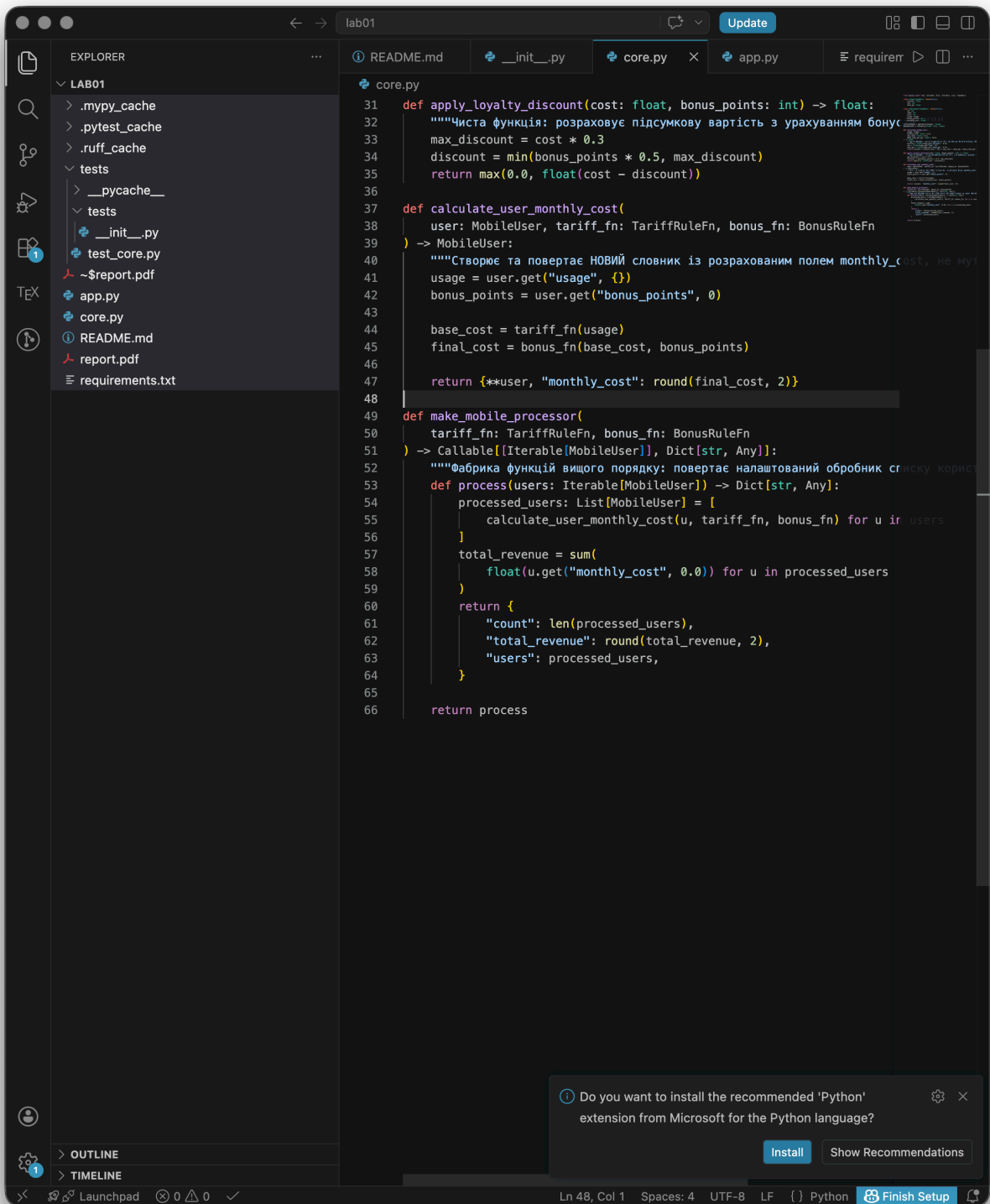
- **Номер та тема варіанта:** Варіант №11 («Мобільний тариф»).

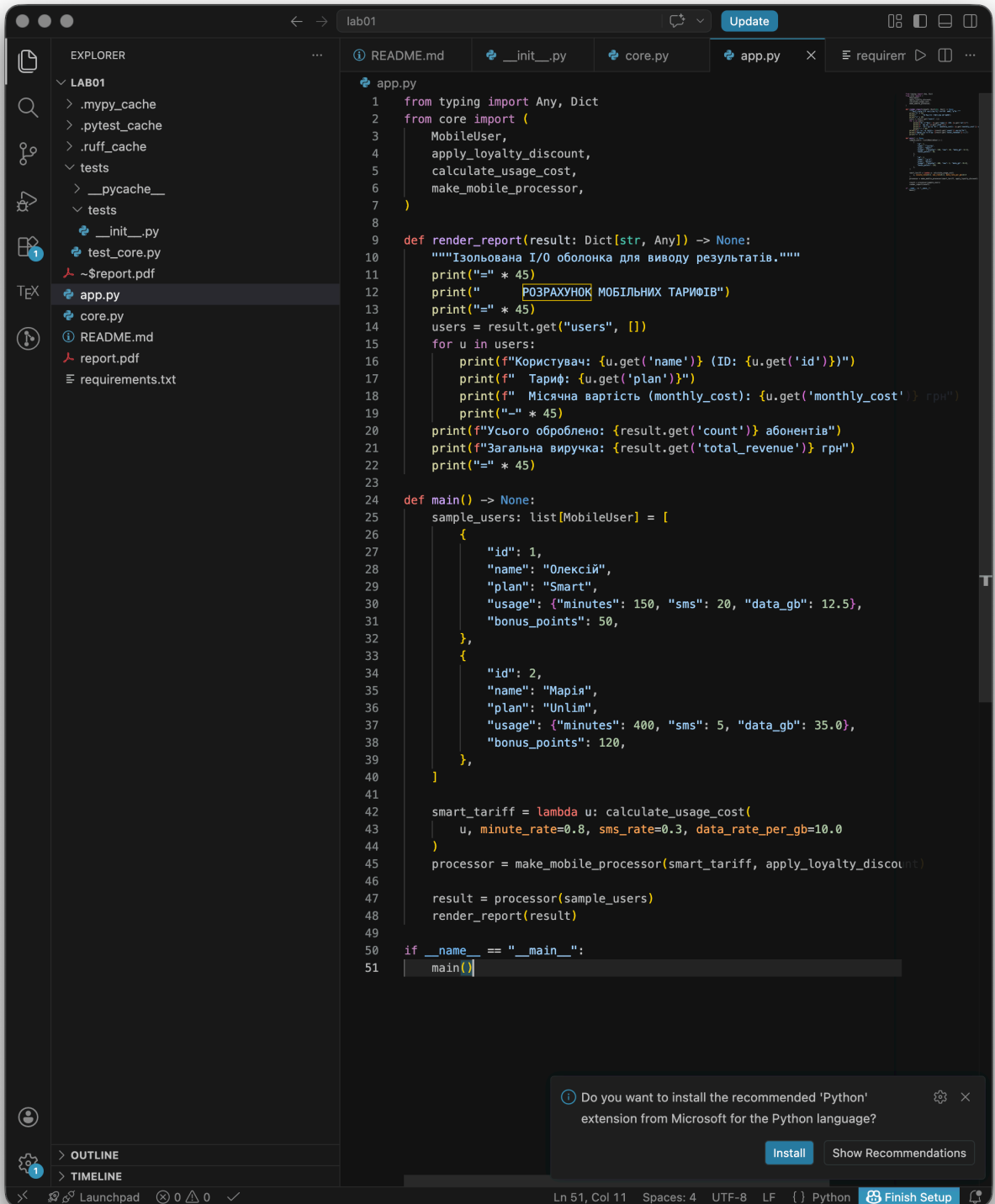
DOCX+ 1

- **Мета роботи:**
 - Зрозуміти, що таке чиста функція і референтна прозорість;
 - Навчитися виявляти та ізолювати побічні ефекти;
 - Переписати імперативний фрагмент у функціональному стилі;
 - Застосувати `typing.Callable` для параметризації обчислень.

ВИКОНАННЯ РОБОТИ







```

from typing import Any, Dict
from core import (
    MobileUser,
    apply_loyalty_discount,
    calculate_usage_cost,
    make_mobile_processor,
)

def render_report(result: Dict[str, Any]) -> None:
    """Ізольована I/O оболонка для виводу результатів."""
    print("=" * 45)
    print("        РОЗРАХУНОК МОБІЛЬНИХ ТАРИФІВ")
    print("=" * 45)
    users = result.get("users", [])
    for u in users:
        print(f"Користувач: {u.get('name')} (ID: {u.get('id')})")
        print(f"    Тариф: {u.get('plan')}")
        print(f"    Місячна вартість (monthly_cost): {u.get('monthly_cost')} грн")
        print("-" * 45)
    print(f"Усього оброблено: {result.get('count')} абонентів")
    print(f"Загальна виручка: {result.get('total_revenue')} грн")
    print("=" * 45)

def main() -> None:
    sample_users: list[MobileUser] = [
        {
            "id": 1,
            "name": "Олексій",
            "plan": "Smart",
            "usage": {"minutes": 150, "sms": 20, "data_gb": 12.5},
            "bonus_points": 50,
        },
        {
            "id": 2,
            "name": "Марія",
            "plan": "Unlim",
            "usage": {"minutes": 400, "sms": 5, "data_gb": 35.0},
            "bonus_points": 120,
        },
    ]

    smart_tariff = lambda u: calculate_usage_cost(
        u, minute_rate=0.8, sms_rate=0.3, data_rate_per_gb=10.0
    )
    processor = make_mobile_processor(smart_tariff, apply_loyalty_discount)

```

```
    result = processor(sample_users)
    render_report(result)

if __name__ == "__main__":
    main()
```

```
from typing import Any, Callable, Dict, Iterable, List, TypedDict

class Usage(TypedDict, total=False):
    minutes: float
    sms: int
    data_gb: float

class MobileUser(TypedDict, total=False):
    id: int
    name: str
    plan: str
    usage: Usage
    bonus_points: int
    monthly_cost: float

TariffRuleFn = Callable[[Usage], float]
BonusRuleFn = Callable[[float, int], float]

def calculate_usage_cost(
    usage: Usage,
    minute_rate: float = 0.8,
    sms_rate: float = 0.3,
    data_rate_per_gb: float = 10.0,
) -> float:
    """Чиста функція: обчислює базову вартість послуг за спожиті хвилини, SMS та трафік."""
    minutes = float(usage.get("minutes", 0.0))
    sms = float(usage.get("sms", 0))
    data_gb = float(usage.get("data_gb", 0.0))
    return minutes * minute_rate + sms * sms_rate + data_gb * data_rate_per_gb

def apply_loyalty_discount(cost: float, bonus_points: int) -> float:
    """Чиста функція: розраховує підсумкову вартість з урахуванням бонусів."""
    max_discount = cost * 0.3
    discount = min(bonus_points * 0.5, max_discount)
    return max(0.0, float(cost - discount))

def calculate_user_monthly_cost(
    user: MobileUser, tariff_fn: TariffRuleFn, bonus_fn: BonusRuleFn
) -> MobileUser:
```

```

        """Створює та повертає НОВИЙ словник із розрахованим полем monthly_cost, не мутуючи
оригінал."""
        usage = user.get("usage", {})
        bonus_points = user.get("bonus_points", 0)

        base_cost = tariff_fn(usage)
        final_cost = bonus_fn(base_cost, bonus_points)

        return {**user, "monthly_cost": round(final_cost, 2)}

def make_mobile_processor(
    tariff_fn: TariffRuleFn, bonus_fn: BonusRuleFn
) -> Callable[[Iterable[MobileUser]], Dict[str, Any]]:
    """Фабрика функцій вищого порядку: повертає налаштований обробник списку
користувачів."""
    def process(users: Iterable[MobileUser]) -> Dict[str, Any]:
        processed_users: List[MobileUser] = [
            calculate_user_monthly_cost(u, tariff_fn, bonus_fn) for u in users
        ]
        total_revenue = sum(
            float(u.get("monthly_cost", 0.0)) for u in processed_users
        )
        return {
            "count": len(processed_users),
            "total_revenue": round(total_revenue, 2),
            "users": processed_users,
        }

    return process

```

Лабораторна робота №1. Варіант 11: Мобільний тариф

Опис проекту

Проект реалізує обчислення місячної вартості мобільного тарифу у функціональному стилі. Чисте ядро ізольоване від побічних ефектів, не використовує глобальний стан та не мутує вхідні дані.

Структура

- `core.py`: Чисте функціональне ядро та типізація.
- `app.py`: Оболонка виводу результатів (I/O).
- `tests/test\_core.py`: Pytest-тести.

Запуск та перевірка

1. Встановлення залежностей:

```

```bash
pip install -r requirements.txt

```

## ВИСНОВКИ

У результаті виконання лабораторної роботи:

- зрозумів, що таке чиста функція і референтна прозорість;
- навчився виявляти та ізолювати побічні ефекти;
- переписав імперативний фрагмент у функціональному стилі;
- реалізував чисте обчислювальне ядро з параметризацією через `typing.Callable` та покрити його unit-тестами.